Project Proposal

Subject: Project Proposal

Project Title: Custom Intrusion Detection System (IDS) & Real-Time Packet Analyzer

Group Members:

Name	Enrollment No.
Aditya Ranjan	24114006
Aditya Pratap Singh Bhadoria	24114005
Satyam Sharma	24114088
Manthan Kothawade	24114050
Om Daddi	24114028
Parth Deokar	24114032

1. The Problem This System Would Solve, and Why It Matters

Every device on a network — a laptop, a router, a phone, a server — talks to other devices by sending small chunks of data called packets. Each packet carries several layers of headers (like nested envelopes inside envelopes) that describe where it came from, where it is going, and what kind of message it carries. Under normal conditions, a network administrator has no easy way to see this traffic in real time, understand whether it is behaving normally, or notice when something is wrong. This is exactly the gap that attackers exploit.

Two very common and very damaging classes of attack illustrate the problem directly:

- **SYN flood / Denial-of-Service attacks:** These attacks abuse the normal three-way TCP handshake by sending a large number of connection-start (SYN) requests without ever completing the handshake, exhausting the target's connection table until it can no longer accept genuine users. This is not a rare or outdated technique — recent industry threat reports from Cloudflare and other network operators show that automated volumetric and protocol-abuse attacks continue to be launched at a rate of tens of millions of incidents per year worldwide, with short, high-intensity bursts increasingly used specifically because they are fast enough to slip past systems that are not actively watching packet-level behaviour.
- **Malformed / spoofed DNS queries:** Because DNS is the system that translates human-readable domain names into machine-readable IP addresses, corrupting or abusing it can silently redirect users to malicious servers, leak information, or be used to amplify a denial-of-service attack. Malformed or oversized DNS queries are a well-documented technique for probing and destabilising poorly-hardened DNS resolvers.

The underlying issue is one of **visibility**. Commercial-grade tools that solve this problem (Wireshark for visibility, Snort/Suricata for detection) exist and are heavily used in the industry, but they are treated by most students as black boxes — used, but never understood. This proposal is important for three concrete reasons:

- **Security relevance:** Network-level attacks remain one of the most common ways organisations are disrupted, and understanding how such attacks look at the packet level is a foundational skill for any network or security engineer.
- **Educational relevance:** Every core topic of the Computer Networks syllabus — framing and error detection, IP addressing, TCP/UDP behaviour, and application-layer protocols — becomes concrete the moment a student watches it happen live on their own machine, instead of reading it as a diagram in a textbook.
- **Practical relevance:** A working, simplified version of this tool gives every group member a tangible, demonstrable artifact that mirrors real tools used in industry SOC (Security Operations Centre) environments, rather than a purely theoretical exercise.

2. What the System Would Do

The system is a lightweight, self-hosted Intrusion Detection System and packet analyzer that observes live network traffic on a machine or a small local network and reports on it in a way a human can understand and act on. At a functional level (not an implementation level), the finished system would provide the following capabilities:

2.1 Traffic capture and visibility

- Observe every packet crossing a chosen network interface in real time, without needing the traffic to be addressed to the observing machine itself.
- Present the observed traffic as a continuously updating, human-readable stream rather than raw bytes.

2.2 Protocol decoding

- Identify and display the structure of each packet's headers across the layers covered in the syllabus: the Ethernet frame, the IPv4/IPv6 header, the TCP/UDP/ICMP header, and — for recognised application traffic — the HTTP, DNS, and FTP contents.
- Detect framing and checksum errors in captured frames, surfacing malformed or corrupted packets instead of silently dropping them.
- Reassemble and present enough of each packet's fields (source/destination address, ports, flags, sequence numbers, payload snippets) that a user can understand exactly what conversation is taking place on the network.

2.3 Threat detection

- Compare live traffic against a configurable set of detection rules (in a format similar to Snort rules), so the set of things the system watches for can be extended without changing the core program.
- Recognise, at minimum, the following classes of suspicious behaviour: SYN floods and other connection-exhaustion patterns, malformed or oversized DNS queries, and other clearly abnormal protocol usage (e.g. unexpected flag combinations, port scans, checksum mismatches).
- Raise a distinct, timestamped alert for every rule match, tagged with a severity and a short human-readable explanation of why it was flagged.

2.4 Reporting and monitoring

- Present live statistics (packets per second, protocol breakdown, top talkers, active alerts) through a real-time dashboard, so a user does not need to read a raw log file to understand the state of the network.
- Maintain a persistent, timestamped log of all flagged events for later review.

Scope boundary (what the system is deliberately not): the system is not a production-grade replacement for Snort/Suricata/Wireshark, does not attempt full protocol coverage of every possible application-layer protocol, and does not attempt to automatically block or mitigate attacks — its job is to observe, decode, and alert, which is exactly the boundary the syllabus topics listed below cover.

Direct syllabus mapping

System Capability	Syllabus Topic Covered
Ethernet framing, checksum/error detection on captured frames	Topic 3 — Framing & Error Detection
IPv4/IPv6 header decoding, source/destination addressing	Topic 5 — IP Addressing
TCP/UDP/ICMP header decoding, SYN-flood and handshake-abuse detection	Topic 6 — TCP/UDP
HTTP, DNS, and FTP payload decoding; malformed DNS query detection	Topic 7 — Application Layer

3. Difficulty Justification

The proposal is scoped to sit deliberately between two failure modes: a toy project that any single student could finish in a weekend, and an industrial-grade tool (matching the full feature set of Suricata or Wireshark) that would be unrealistic for a semester. The following points justify why this sits in the right band of difficulty for a 6-member group over one semester.

3.1 Why it is not too easy

- It requires correctly decoding **multiple, layered protocol formats** (Ethernet, IPv4/IPv6, TCP/UDP/ICMP, HTTP/DNS/FTP) directly from raw bytes — each layer has its own header format, byte ordering rules, and validity checks, so a single mistake in an earlier layer breaks every layer built on top of it.
- It requires handling a **live, high-throughput, real-time data stream** rather than a static input file — packets can arrive faster than they can be processed, which forces the group to actually confront performance and correctness together, not just correctness in isolation.
- It requires designing a genuine **rule-matching / pattern-detection system** that can flag abnormal behaviour (e.g. distinguishing a real SYN flood from a burst of legitimate traffic) — this is a non-trivial state-tracking and classification problem, not a simple lookup.

3.2 Why it is not too difficult

- The scope is bounded to a small, well-defined set of protocols and a small, well-defined set of attack signatures — the group is not attempting to cover the entire protocol universe or defend against every known attack class, only the ones directly tied to the syllabus.
- Mature, well-documented open building blocks exist for the lowest-level, most tedious part of the problem (raw packet capture), so the group's effort is concentrated on the parts that are genuinely educational — decoding, rule matching, and presentation — rather than on solving operating-system-level driver problems.
- The system deliberately stops at detection and alerting; it does not attempt automated mitigation, traffic shaping, or active defence, which removes an entire, much harder category of work.

3.3 Fit for a group of this size

The project naturally decomposes into six largely independent components — capture, lower-layer parsing, upper-layer parsing, application-layer parsing, rule matching, and logging/UI — each of which is substantial enough to be one group member's primary contribution, and each of which has a clean, well-defined interface (a packet or parsed-packet structure) to the components on either side of it. This means the six-person group can work in parallel for most of the semester with minimal blocking dependencies, while still needing to integrate their work into a single coherent system by the end — which is itself a realistic and valuable part of the exercise.